

Unity Client Programmer

이예빈

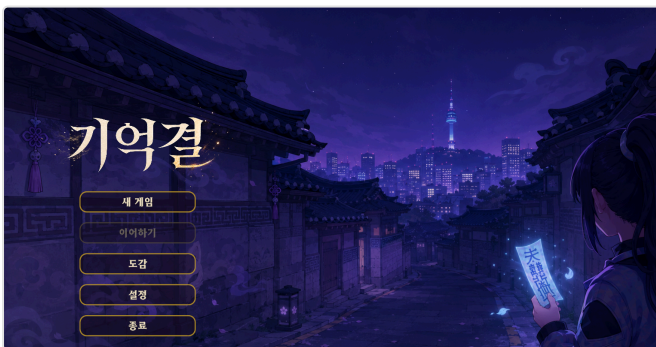
YEBIN LEE

다양한 Unity 기반 게임 프로젝트를 제작하는 데 익숙하고,
게임 제작에 AI를 적극적으로 활용하는 데에 자신이 있습니다.

제1회 서울 플레이업 AI 게임 챌린지에 참가해 턴제 덱빌딩 로그라이크 기억결(MemoryStone)을 개발하고
있습니다. 상위 8팀에 선정되어 2026 GES 전시를 앞두고 있으며, 2027년 상반기 Steam 출시를 목표로 개발
을 이어가고 있습니다.

Email puding2564@gmail.com

Phone [010-9473-7840](tel:010-9473-7840)



기억결 — Unity 6 · PC(2026) · Rainbow War — Unity 2020.3 · Android(2022)

기억결 / MemoryStone		
01	인게임 플로우	게임 소개와 한 턴의 진행 순서
02	도메인 코어와 어댑터 분리	게임 로직을 UnityEngine 비의존 계층으로 분리한 과정
03	데이터 중심 구조	CSV 저작 → 에디터 임포터 → ScriptableObject 파이프라인
04	Unity Profiler 기반 성능 최적화 (1)	진단 과정 · 시야 갱신 · 몬스터 예고 오버레이
05	Unity Profiler 기반 성능 최적화 (2)	조명 마스크 재빌드 · 씬 로드 · 결과
Rainbow War		
06	영역 확장 · 침탈 알고리즘	폴리곤을 폴리라인으로 자르는 확장 규칙과 침탈 시 조각 분리
07	Photon 멀티플레이 동기화	호스트 권위 판정 · 시작 시점 동기화 · 레이트 조인 복원
그 외		
08	기타 프로젝트	설화 · Match & Mist

서울의 랜드마크와 전통 요괴를 배경으로 한 **턴제 덱빌딩 로그라이크**입니다. <Slay the Spire> 같은 덱 빌딩 로그라이크 장르에 **타일 맵 JRPG** 장르를 합친 구조입니다. 육각 타일 맵 위에서 플레이어는 **주변 3칸 시야**의 정보와 카드를 적절하게 사용하여 어둠 속에 존재하는 요괴를 물리치며 서울을 탐험합니다. 카드가 공격·방어만이 아니라 **이동과 경찰**도 수행하기 때문에, “무엇을 살까”와 “어디로 갈까”를 같은 손패로 동시에 고민하게 만드는 것이 이 게임의 차별점입니다.



전투 — 육각 타일 위에서 요괴와 교전. 붉은 칸이 몬스터의 공격 범위 예고



도감 — 카드 · 상태이상 · 유물 · 함정 · 몬스터를 한곳에서 열람

프로젝트

팀	얼갈이 게임즈 · 3인 · 팀 리더
기간	2026.05 ~ 진행 중
엔진	Unity 6000.3
성과	제1회 서울 플레이업 AI 게임 챌린지 상위 8팀 · 2026 GES 전시 예정
계획	2027년 상반기 Steam 출시 목표
AI 도구	Claude Code · Codex · Unity MCP · VARCO 3D · VARCO Sounds

담당 범위

개발	클라이언트 · 게임 로직 전담
기획	게임 기획 전반 · 레벨 디자인
UI/UX	HUD · 카드 레인 · 상점 · 보상 · 튜토리얼
아트	카드 · 아이콘 2D 일러스트 생성 · VFX 재창작
데이터	CSV 저작 → 에디터 임포터 파이프라인
품질	자동화 테스트 · 성능 계측 인프라
배경 · 캐릭터 모델링	아트는 팀원이 담당했습니다.

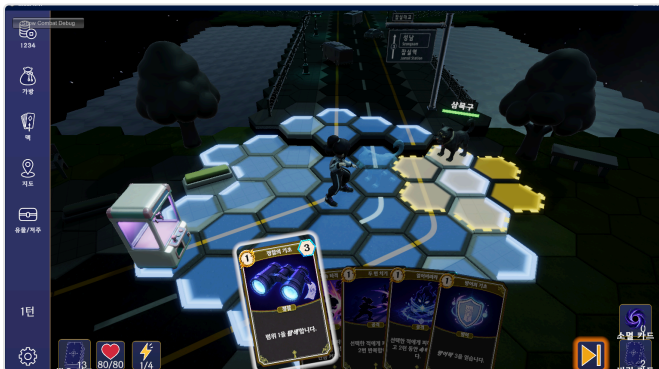
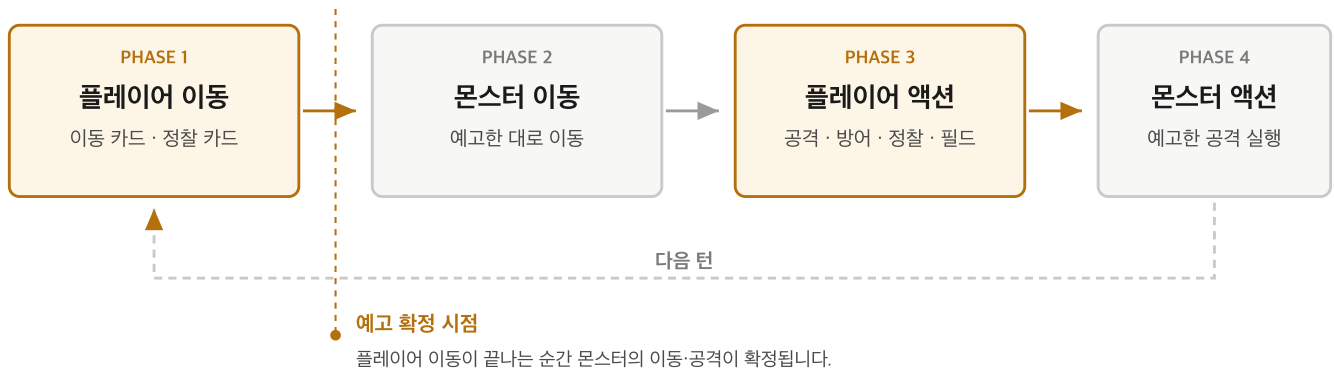


기억결 트레일러 · 플레이 영상
youtu.be/81n_Go-B73c

직접 설계 · 구현한 시스템

아키텍처 설계	게임 로직을 UnityEngine 비의존 도메인 코어로 분리하고 Unity를 어댑터로 배치	가 보지 않은 지역을 가리는 안개. 화면에서 어둡게 칠하는 대신 UI가 읽을 데이터를 자체를 차단하고, 시야는 조명 마스크로 표현
헥스 좌표계 · 거리 로직	축좌표계로 이웃 · 거리 · 회전을 정수 연산화. 직선 거리 · Dijkstra · BFS를 용도별로 분리	헥스 맵을 데이터 세트로 저장하는 Editor 확장. 도구 14종 · 스트로크 Undo · 저작 검증 리포트
턴 페이지 시스템	한 턴을 이동 · 액션 네 단계로 나누고, 몬스터의 다음 행동을 플레이어가 이동이 끝난 시점에 확정해 예고한 대로만 실행되게 보장	위험 예산과 가중 추첨으로 몬스터 · 함정 배치. 시드를 저장해 같은 판을 재현
데이터 중심 구조	카드 · 몬스터 · 상태이상을 CSV로 저장하고 에디터 임포터가 ScriptableObject로 변환	자동화 테스트 회귀 게이트 EditMode 테스트 925건과 렌더 복잡도 기준값 대조로 성능 · 동작 회귀를 자동 검출
연출 재생 상태 관리	연출 타이밍을 이펙트 버퍼에 쌓아 애니메이션 비트에 맞춰 재생. 전용 설정 씬을 만들어 전투 없이 VFX · SFX 타이밍을 조정	아트 가이드 · 프리셋 · 데이터 행을 읽어 프롬프트를 산출하는 컴파일러. 가끔 게이트와 생성 출처 기록

게임은 턴제 방식으로 진행되며 한 턴은 **플레이어 이동 → 몬스터 이동 → 플레이어 액션 → 몬스터 액션** 네 페이즈로 돌아갑니다. 이동과 액션이 나뉘어 있어, **먼저 자리를 잡고 나서 무엇을 할지 고르는** 순서가 됩니다. **기력**이라는 공유 자원으로 이동과 액션(공격 · 방어)을 분배하기 때문에, 플레이어는 이동과 액션을 잘 분배하며 탐험과 전투를 진행합니다.



카드 조작 화면 — 손패에서 카드를 골라 이동 · 정찰 · 공격 · 방어한다



몬스터의 이동 경로와 공격 범위가 칸에 미리 표시된다



플레이어 시야는 주위 3칸으로, 그 밖의 정보는 가려져 있다



전투 후 카드를 골라 덱을 키운다

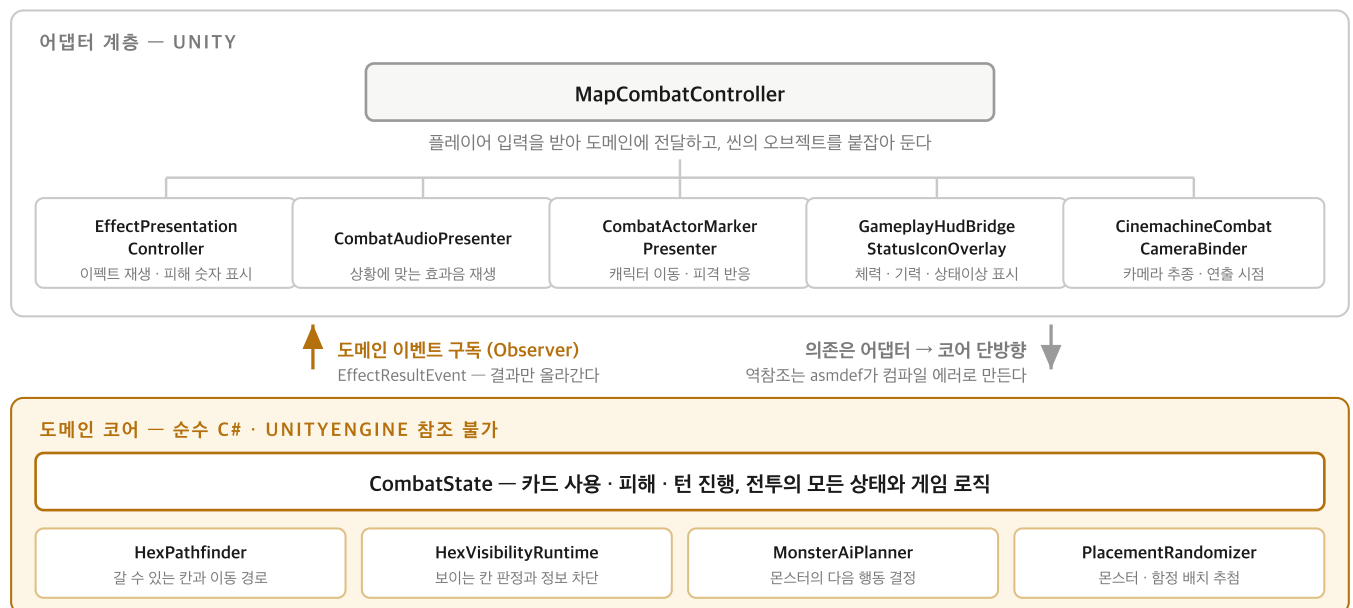
현재 구현 범위 — 카드 60종 · 몬스터 6종 · 레벨 2개(튜토리얼 맵, 스테이지 맵)

문제

- ① 초기에는 게임 로직이 Unity Engine에 종속되어, 로직을 추가하거나 변경할 때마다 **Play 모드에 진입해 테스트**해야 했고 해당 시간이 개발 속도를 늦췄습니다.
- ② AI 에이전트를 개발에 투입하면서 **테스트 단계에서 사람이 병목**이 되었습니다. 로직의 작성 속도를 사람이 따라가기 벅찼고, 규모가 커질수록 **작성한 기능의 테스트가 자주 미뤄지는** 문제가 발생했습니다.
- ③ 초기에는 하나의 MonoBehaviour 파일에 모든 게임 로직을 작성했고, 그 결과 한 파일이 **10,690줄**까지 커졌습니다.

선택한 구조

- 도메인 코어 / 어댑터 분리 — Ports & Adapters(헥사고날)**
전투 · 맵 · 카드 로직을 UnityEngine 비의존 어셈블리로 떼어내, **Play 모드 없이 EditMode와 CLI 환경에서** 테스트를 검증할 수 있도록 했습니다. → ① ②
- 결과 이벤트 발행 — Observer**
도메인은 결과만 이벤트로 올리고 화면 · 소리 · 카메라가 각자 구독하도록 하여, 이후 **연출을 늘려도 도메인 코드가 커지지 않게** 했습니다. → ③
- asmdef 참조 제약**
어댑터 → 코어 단방향만 허용하여 빌드를 막는 조건을 두었습니다.



효과

얻은 것

근거

① Play 모드 없이 게임 로직 검증

씬 로드와 Play 모드 진입 없이 에디터에서 바로 실행되는 **EditMode 테스트 925건**.
로직 변경 후 확인까지 초 단위

② AI 에이전트가 검증까지 스스로 수행

도메인이 UnityEngine 비의존이라 **CLI 테스트 환경 조성**

③ 연출과 게임 로직 파일의 비의존성 확보

이펙트 · 효과음 · 화면 흔들림 · 카메라 연출을 **Presenter와 CSV 기반 큐 테이블**에서 추가

문제

- ① 공모전 1차 심사를 위해 **한 달이라는 한정된 기간** 안에 제작해야 했고, 제작 도중 기획을 수정하는 일이 많았습니다. 단순 serialized field로 데이터를 관리해서는 **제작 속도 · 협업 · 밸런싱**에 어려움이 있었습니다.
- ② ‘카드 게임’이라는 특성상 **데이터가 많고 효과가 다양해** 관리가 어려웠습니다. 또한 카드에 따라 다양한 규칙이 존재하는데, **별도의 검증 루프가 없어 오류가 자주 발생**했습니다.

해결 방안

- CSV 저작 → 에디터 임포터 → ScriptableObject**
수치 · 사거리 · 코스트 · 희귀도까지 전부 컬럼으로 옮겨, 인스펙터를 거치지 않고 **표 한 장에서 밸런싱**합니다. → ①
- 효과 디스패처 (전략 패턴)**
공통 카드는 공통 경로로 처리하고, 규칙이 다른 카드만 전용 핸들러에 위임해 **카드가 늘어도 분기가 늘지 않게** 했습니다. → ②
- 임포트 시점 검증**
잘못된 값을 관대하게 넘기지 않고 **임포트를 실패**시켜, 게임을 켜기 전에 검증 루프가 돌게 했습니다. → ②



효과 디스패처 — 특수 카드만 핸들러를 가진다

```
// Combat/Runtime/CombatState.CardEffects.cs
defendHandlers =
    ToEffectRefMap<IDefendEffectHandler>(new[]
    {
        new ZeroThenDoubleDefendEffectHandler(),
        new HalfReflectDefendEffectHandler(),
        new CleanseBlockDefendEffectHandler(), ..
    });
```

카드 종류는 이동 · 방어 · 유틸 · 정찰 · 공격 · 필드 6가지 타입으로 나뉘어 있고, 이와 별개로 **특수 카드 핸들러 24종**을 구현했습니다.

일반 카드는 타입별 공통 경로를 그대로 타고, 규칙이 다른 카드만 **effectRef**로 자기 핸들러를 찾아갑니다.

효과

① 밸런스 조정에 빌드 불필요

데이터 수정 시 CSV 행을 고치고 임포트하여 반영. 기획 담당 팀원이 수치와 설명 문구를 직접 수정

② 카드 추가가 기존 카드에 영향 없음

기존에 없던 규칙일 때만 핸들러를 하나 추가하고 화이트리스트에 등록

② 잘못된 데이터가 게임에 도달하지 않음

임포트가 먼저 실패. 규칙 자체도 임포터 테스트가 검증

적용 범위 — 카드 · 몬스터 · 공격 패턴 · 보스 페이지 · 상태이상 · 유물 · 소모품 · 상점가 · 보상 · VFX · 사운드 · 공격 타이밍까지 **CSV 원본 27개**가 같은 파이프라인을 씁니다.

공모전 일정에 맞춰 기능을 우선하여 프로토타입을 제작한 후, 성능 최적화를 진행하였습니다. 그 당시 **턴이 전환될 때마다 눈에 띄는 속도 저하**가 있었고, 이를 해결하기 위해 여러 진단과 테스트를 진행했습니다.

진단과 테스트

① 턴 전환 구간 측정

steady : 8.8 ms draw call 205
 move_turn : 56.0 ms draw call 210
 → 그릴 양은 그대로, 시간만 6배

② ProfilerMarker 로그

[턴 전환 기준]

Vis.ApplyVisibility 59.0 ms ← 암시야 처리
 Vis.OverlayApply 1.2 ms
 Pathfind 4.8 ms

[이동 1회 기준]

Vis.OverlayBuild 10.4 ms ← 몬스터 예고

- ① GPU vs CPU — 가장 먼저 Profiler로 턴이 전환되는 타이밍의 **드로우콜과 지연 시간**을 측정했습니다. 드로우콜은 큰 변화가 없었으나 지연 시간만 **8.8 → 56.0 ms** 수준으로 폭발하는 것을 확인했고, **CPU의 턴 처리 코드**가 원인이라고 파악했습니다.
- ② 어느 함수인가 — 턴 처리 경로에 지연 시간을 기록하는 ProfilerMarker를 제작했습니다. 로그를 확인한 결과 **암시야 처리 로직**과 **몬스터 예고 오버레이** 두 곳이 원인임을 확인했습니다.

CASE 1 시야 갱신

더티 플래그 · 증분 갱신

0(n) → 0(k)

59 → 5.8 ms/turn

초기 구현

시야가 바뀌면 **맵 전체의 안개 체크 메시지를 파괴하고 다시 생성**했습니다. 스테이지 한 판의 타일이 약 **4,500칸**인데, 몇 칸이 바뀌었든 항상 전체를 훑습니다. 게다가 이 갱신이 턴당 두 번 돌고 있었습니다.

측정

Vis.ApplyVisibility
 59 ms/turn

턴 전환 지연의 대부분

조치

칸별 (Exists, Visibility)를 기억해 두고 **직전과 달라진 칸만 재적용**합니다. 안개 메시도 **미탐색·탐색 칸 집합이 실제로 바뀌었을 때만** 재생성합니다.

여기에 **버전 스탬프**를 얹어, 호출자가 “바뀐 것 없음”을 알리면 맵 전체 재스캔을 통째로 건너웁니다. 선택만 바꾸는 갱신은 바뀐 칸이 0이므로 **낭비되던 두 번째 패스가 함께 사라졌습니다**.

한 번 이동에 실제로 달라지는 칸은 시야 경계 부근 **수십 칸**입니다. 전체 칸 **n**을 매번 훑던 작업이 바뀐 칸 **k**에 비례하도록 바뀌었습니다.

CASE 2 몬스터 예고 오버레이

메모이제이션 · 해시 무효화

0(m) → 0(1)

10.4 → 2.1 ms/move

초기 구현

몬스터의 이동 예고 · 공격 범위 · 추격 표시를 **화면을 갱신할 때마다 처음부터 다시 계산**했습니다. 같은 상황에서 같은 결과를 반복해 만들고 있었습니다.

측정

Vis.OverlayBuild
 10.4 ms/move

한 칸 이동할 때마다 지불하는 비용

조치

흩어져 있던 계산을 하나로 합치고, **결과를 캐시**했습니다. 다시 계산하는 조건은 “**상황이 실제로 바뀌었는가**” 하나입니다.

여기서 ‘상황’은 몬스터 위치 · 예측 이동 · 공격 패턴 · 플레이어 위치 · 가시성을 **하나의 해시 값으로 압축**한 것입니다. 이전 해시와 같으면 계산을 건너뛰고 이전 결과를 그대로 씁니다.

▼ **재측정** — CASE 2까지 적용한 뒤, 정본 썬의 암시야 표현을 오버레이 토티에서 **조명 마스크 모드**로 전환했습니다. 그러자 시야 갱신 비용이 다시 올라갔고, 마커를 따라가 원인을 좁혔습니다.

CASE 3 조명 마스크 재빌드

좌표 변환 메모이제이션

 $O(R \cdot n) \rightarrow O(n)$ 1.97 → **0.18 ms/frame**

초기 구현

조명 마스크 방식 — 맵 위에 검은 밝기 텍스처를 한 장 없고, 셰이더가 타일을 그릴 때 자기 위치의 픽셀을 읽어 색에 곱합니다. 픽셀이 검은면 그 타일은 어둡게 보입니다. 크기는 $128 \times 128 =$ 픽셀 16,384개입니다.

시야는 한 칸 움직일 때마다 바뀌므로, 이동할 때마다 픽셀 16,384개를 다시 채웁니다.

```
for i in 0..16383:
    칸 = WorldToCoord(i)
    pixels[i] = 밝기[칸]
```

WorldToCoord를 매번 다시 계산합니다.

측정

Atlas.Vis.LightingMask

1.97 ms/frame

60fps 예산 16.7 ms의 약 12%를 마스크 재빌드 하나가 쓰고 있었습니다.

조치

WorldToCoord를 메모이제이션했습니다. “픽셀 i는 어느 칸인가”의 답은 맵과 텍스처 크기가 고정이라 게임 내내 같은 값입니다. 매번 바뀌는 것은 그 칸의 밝기뿐입니다.

```
// 1회만 계산해 배열에 저장
칸번호[i] = WorldToCoord(i)

// 이동할 때마다
pixels[i] = 밝기[칸번호[i]]
```

좌표 변환 횟수가 이동할 때마다 16,384회에서 게임 전체 통틀어 16,384회로 바뀝니다. 재빌드에 남는 것은 배열 조회뿐입니다.

계산 결과를 바꾼 게 아니라 계산 시점을 앞으로 옮긴 것이라 출력 픽셀이 이전과 동일합니다.

▼ **별개 증상** — 위 셋은 전부 프레임 시간이었습니다. CASE 4는 **플레이 시작 전 대기**로 관찰돼, 프레임 마커가 아니라 썬 파일 크기와 로드 시간을 직접 재는 것에서 출발했습니다.

CASE 4 썬 로드

직렬화 데이터 감축

Raycast → $O(1)$ 역산17.5초 → **0.56초**

초기 구현

타일 클릭을 받으려고 타일마다 콜라이더를 붙이고, 마우스 위치에서 Raycast를 쏘아 어떤 타일에 맞았는지 판정했습니다. 가장 단순하고 확실한 방법이었지만, 타일과 그 측면마다 콜라이더가 붙어 맵 하나에 클릭 콜라이더 139,146개가 썬 파일에 그대로 구워졌습니다.

측정

썬 파일 317 MB

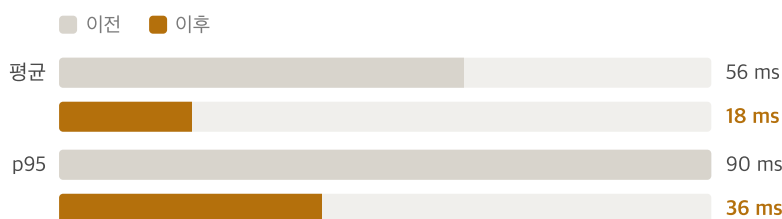
로드 17,480 ms

플레이를 늘려도 17.5초 동안 아무것도 시작되지 않는 상태

조치

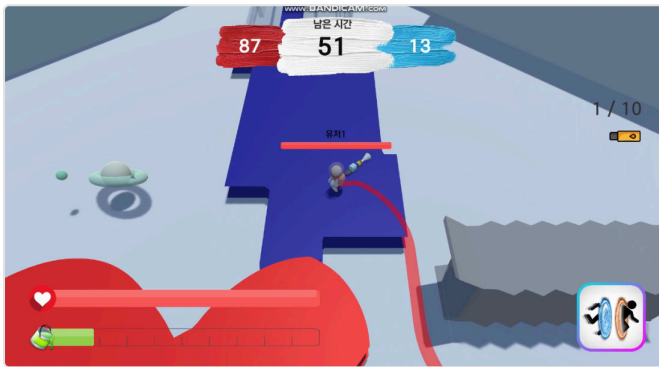
콜라이더와 Raycast를 걷어내고, 스크린 좌표를 hex 좌표로 직접 역산해 클릭을 판정하도록 바꿨습니다. 물리 표면 없이 순수 계산으로 처리됩니다. 썬은 50 MB가 됐습니다.

턴당 프레임 비용 (move_turn, ms) — 60fps 예산 16.7 ms



결과 — 턴 이동 시 프레임 56 → 18 ms(p95 90 → 36 ms), 썬 로드 17.5초 → 0.56초. 개선은 전부 회귀 게이트로 잠가 되돌아가지 않게 했습니다.

슈팅과 **영역 점령(땅따먹기)**을 결합한 **모바일 멀티플레이 TPS**입니다. 자기 영역 밖으로 나가 선을 긋고 돌아오면 그 선이 감싼 만큼 땅이 넓어지고, 상대 영역을 가로지르면 그만큼 상대 땅을 깎습니다. 제한 시간 안에 더 넓게 칠한 팀이 이깁니다. 선을 긋는 동안은 무방비여서, **총을 쏘는 시간과 땅을 넓히는 시간을 맞바꾸는 것이** 이 게임의 축입니다. 7인 팀에서 **개발 리더**를 맡아 **실시간 멀티플레이 전반과 영역 시스템 전체**를 담당했습니다.



퀘적을 긋는 중 — 빨강이 자기 영역, 파랑이 상대 영역. 긋는 동안은 무방비다



확정된 폴리곤을 삼각분할해 만든 영역 메시. 상단은 양 팀 점유율과 남은 시간

프로젝트

팀	아주대 게임 개발 동아리 · 7인
기간	2021.04 ~ 2022.04
엔진	Unity 2020.3.26f1
플랫폼	Android · 가로 고정
네트워크	Photon PUN2 v2.40 (Photon Cloud)
성과	교내 동아리 시연회 출품
소스	github.com/binnini/Rainbow-War

담당 범위

영역	확장 · 침탈 알고리즘, 퀘적 기록, 런타임 메시 생성 전담
멀티	로비 · 룸 매칭, 상태 동기화, 호스트 권위 판정
인게임	시작 배리어 · 타이머 · 점수 · 사망 · 리스폰
UI/HUD	체력바 · 이름표 추적, 팀 점수판, 룸 플레이어 리스트
입력	PC 즉시 입력을 모바일 토글 상태 입력으로 재설계
사격 · 조준 · 총알	팀원이 담당했습니다.



Rainbow War 플레이 영상
youtu.be/I2qebtkdKGU

직접 설계 · 구현한 시스템

영역 확장	퀘적이 나눈 두 갈래로 조각 2개를 만들고 넓이가 넓은 쪽을 채택. 폴리곤을 합치는 기하 연산 없이 넓이 비교만으로 확장을 끝냄
영역 침탈	영역을 조각 목록으로 표현하고, 자를 때는 어느 쪽도 버리지 않은 뒤 마지막에 조각별 소유권을 한 번씩 판정
선 밝기 판정	렌더링 전용인 퀘적 위에 총돌체를 간격 제어하며 부착. 최근 2개를 비활성으로 뒤 자기 꼬리 오탐 차단
영역 동기화	계산은 소유 클라이언트에서만 수행하고, 확정된 정점 배열만 폐쇄 · 침탈 시점에 전송
시작 시점 동기화	서버 동기 시계의 원점을 룸 프로퍼티로 공유하고, 오브젝트 생성 완료 배리어로 초기화 순서 보장

영역은 플레이어가 궤적을 완성할 때마다 갱신됩니다. 이때 자신의 영역이 넓어지는 **확장**과 상대방의 영역을 제거하는 **침탈** 두 가지 효과가 일어납니다. 여기서 ① **확장 시 어느 갈래를 남길지**, ② **침탈 결과가 여러 조각으로 갈라지는 것** 둘을 풀어야 했습니다.

① 확장 둘레 두 갈래 중 어느 쪽을 남길 것인가

폴리곤 합집합 연산 없음



궤적이 영역을 떠난 A와 되돌아온 B가 기존 폴리곤 둘레를 두 갈래로 나눕니다. 양 끝점을 가장 가까운 정점에 스냅한 뒤, 각 갈래에 궤적을 이어 붙여 조각 2개를 만듭니다.

두 조각 중 **넓은 쪽**이 곧 '기존 영역 + 새로 감싼 땅'입니다. 궤적은 영역 밖에 있을 때만 기록되므로, 새로 감싼 땅은 기존 영역과 겹치지 않습니다. 그래서 두 폴리곤을 합치는 기하 연산 없이 넓이만 비교해 확장을 끝냅니다.

```
// Trail_Action.ExpandArea
TrailArea.TryCut(areaPieces[i], trail,
    out a, out b);
areaPieces[i] = GetPolygonArea(a)
    >= GetPolygonArea(b) ? a : b;
```

② 침탈 영역이 상대에 의해 잘리면, 잘린 만큼 복수의 폴리곤으로 나뉜다



상대 영역을 가로질러 지나가면 그 영역이 **내 궤적을 경계로 잘립니다**. 가져오는 것은 **내가 감싼 쪽**이고, 그 조각은 좁을 수도 넓을 수도 있습니다. ①의 넓이 규칙은 여기서 쓸 수 없습니다.

한 궤적이 상대 영역을 여러 번 드나들면 상대에게는 **조각 여러 개가 남습니다**. 그래서 영역을 폴리곤 하나가 아니라 `List<List<Vector3>>` 조각 목록으로 표현합니다.

```
foreach (var cut in cuts) // 다 자르고
    pieces = CutAll(pieces, cut); // 둘 다 보관
foreach (var p in pieces) // 판정은 마지막에
    if (!IsTakenBy(p, this, cutPoints))
        survived.Add(p);
```

자를 때는 어느 쪽도 버리지 않고, 전부 자른 뒤 **조각마다 소유권을 한 번씩 판정**합니다.

출처 표기 — 포함 판정(Ray casting) · 넓이(Shoelace) · 삼각분할(Ear clipping)은 공개 구현을 가져다 쓰고 코드 주석에 출처를 남겼습니다. 제가 설계한 것은 이를 조합한 **영역 표현과 확장 · 침탈 규칙**입니다.

Photon PUN2(Photon Cloud) 기반입니다. 방·팀 매칭과 인게임 상태 동기화를 담당했고, 실제로 여러 대를 붙여 돌리면서 겪은 두 가지 문제를 해결했습니다.



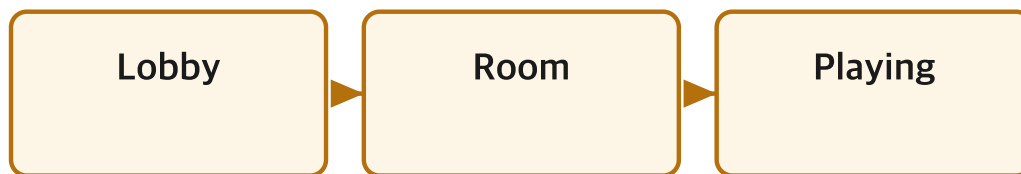
로비 — 마스터 서버 접속 상태



룸 — 팀 배정과 색 선택



처치 · 리스폰 — 점수는 마스터가 계산



랜덤 입장 → 실패 시 방 생성 팀 · 색 검증을 통과해야 시작 마스터 기준 씬 자동 동기화

시작과 동시에 IsOpen = false 로 난입 차단

구현 내용 — Photon PUN2를 어디에 썼는가

JoinRandomRoom · CreateRoom	로비 매치메이킹. 랜덤 입장에 실패하면 방을 만든다
PhotonTeamsManager CustomProperties	룸의 팀 배정과 색 선택 상태를 참가자 전체에 공유
AutomaticallySyncScene	마스터 기준으로 Playing 씬을 동시에 전환
RPC (MasterClient · Others)	체력 · 점수 · 킬 크레딧을 마스터에서만 계산하고 결과만 브로드캐스트
PhotonNetwork.Time	방 전체가 공유하는 서버 시계. 제한 시간 계산의 기준

영역 동기화 — 계산은 로컬, 결과만 이벤트로

폴리곤 절단은 무거운 연산이라 매 프레임 스트리밍할 수 없습니다. 소수 클라이언트에서만 계산하고, 확정된 정점 배열만 패쇄 · 침탈 시점에 보냅니다.

겪은 문제와 해결

① 클라이언트마다 시작 시점이 어긋난다

문제 — 기기와 회선 차이로 씬 로딩 완료 시각이 다릅니다. 각자 타이머를 돌리면 60초 제한 시간이 서로 다른 시점에 끝나 승패 판정이 어긋납니다.

해결 — 마스터가 서버 시계의 원점을 룸 프로퍼티에 기록하고 나머지가 그 값을 읽어 같은 경과 시간을 계산합니다. RPC와 달리 룸 프로퍼티는 **상태로 남아 늦게 읽어도 유효합니다**. 여기에 오브젝트 생성 완료 배리어를 더해 초기화 순서도 보강했습니다.

② 늦게 들어온 클라이언트가 남의 영역을 모른다

문제 — 영역은 변경 시점에만 브로드캐스트되므로, 나중에 들어온 클라이언트는 그 전의 이벤트를 전부 놓칩니다.

해결 — 진입한 쪽이 기존 참가자에게 최신 영역을 역요청(pull)하도록 했습니다. OthersBuffered 로 push하면 게임 내내 모든 변경 이력이 버퍼에 쌓입니다.

지금이라면 고칠 것 — 점수판을 Update 에서 매 프레임 전 클라이언트에 RPC로 보내고 있습니다 (당시에도 비효율을 알고 코드에 TODO를 남겨뒀습니다). 영역도 조각 전체를 보내므로 정점 수에 비례해 페이로드가 늘어납니다. 당시에는 **전송량과 프레임 시간을 측정하지 않았**습니다.

설화 — 조선 배경 횡스크롤 던제 로그라이크

개발 전담 · 4인 · 2022.04 ~ 2022.12



마을 — 상점 · 게시판 등 던전 밖 진행 흐름과 UI를 구현했다

- 속도 순으로 턴이 도는 파티 단위 턴제 전투, 캐릭터별 고유 기술과 상태이상 구현
- 캐릭터 · 스토리 · 이벤트를 JSON 데이터로 분리해 콘텐츠 추가 시 코드를 고치지 않아도 되게 구성
- 전투 이벤트 전파에 Observer, 생성에 Factory 패턴을 적용해 8개월간 늘어나는 요구사항에 대응



보스전 진입 컷씬 — 마을 · 상점 · 던전을 오가는 구조



설화 플레이 영상

youtu.be/swJEF2gJw50

Match & Mist — 제한 시야 추적 게임

Global Game Jam 2022 · 3일



성냥불로 안개를 밝히며 좀비를 피한다

- 주제 '이중성'에 맞춰 아이템으로 도망과 추적이 뒤바뀌는 규칙을 핵심으로 설계
- 이동 속도가 소음이 되고 좀비가 그 소음을 추적하는 제한 시야 추적 구현
- 3일 안에 플레이 가능한 빌드를 완성해 제출



이동 속도가 소음이 되고, 좀비가 그 소음을 추적한다



Match & Mist 플레이 영상

[drive.google.com/file/d/](https://drive.google.com/file/d/1VsZmacooxUqKX8ptluRXxg7rhBjFIJOL)[1VsZmacooxUqKX8ptluRXxg7rhBjFIJOL](https://drive.google.com/file/d/1VsZmacooxUqKX8ptluRXxg7rhBjFIJOL)